Beyond Code Generation

Give AI Eyes and Hands for .NET MAUI

.NET

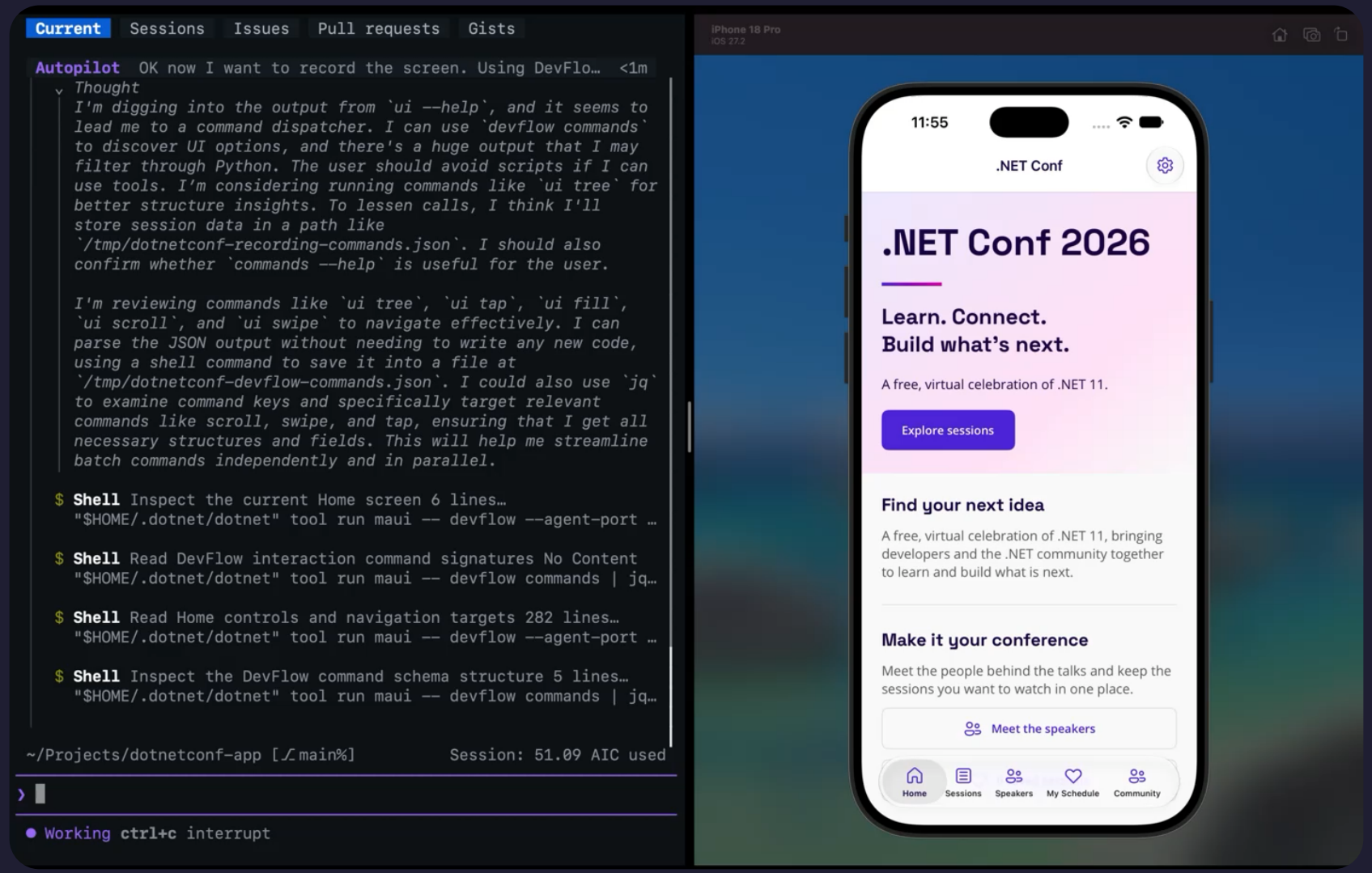


Gerald Versluis

Software Engineer, Microsoft

Your AI writes the XAML.
It has never seen the app run.

Demo: Copilot drives the .NET Conf app



Four tools from maui-labs

MAUI CLI

Sets up SDKs, emulators and devices

- **maui doctor** finds and fixes environment issues
- Installs Android SDK, JDK and emulators
- Manages Xcode, simulators and runtimes
- **--json** output for CI pipelines

```
maui doctor
```

AI skills

Teach your agent how MAUI works

- Installs MAUI and DevFlow skills
- Adds Copilot agents and MCP config
- Targets VS Code, Copilot CLI and Claude
- **--dry-run** previews every change

```
maui ai init
```

DevFlow

Lets the agent see and drive the app

- Visual tree, taps and screenshots via an in-app agent
- Network monitoring and profiling
- Blazor WebViews over Chrome DevTools
- MCP server lets AI agents test your app

```
maui devflow
```

Inspector

A live view you share with the AI

- Live screenshot of the app
- Visual tree of every control
- Properties, layout and bindings
- Logs, network and diagnostics

```
Browser·VS Code·Copilot
```

01 Set up



One tool, every device

Checks your machine and fixes what is missing.

Manages SDKs, emulators and simulators.

Your agent uses the same commands.

dotnet build, run, test and watch now work with MAUI too. **See Jonathan Peppers' session.**

```
# Install once
dotnet tool install -g Microsoft.Maui.Cli --prerelease

# Check and fix your environment
maui doctor

# See every device and emulator
maui device list

# Machine-readable output for agents
maui doctor --json

# Build and run stay with dotnet
dotnet run -f net11.0-android
```

MAUI Sherpa

Your whole MAUI setup in one desktop app.

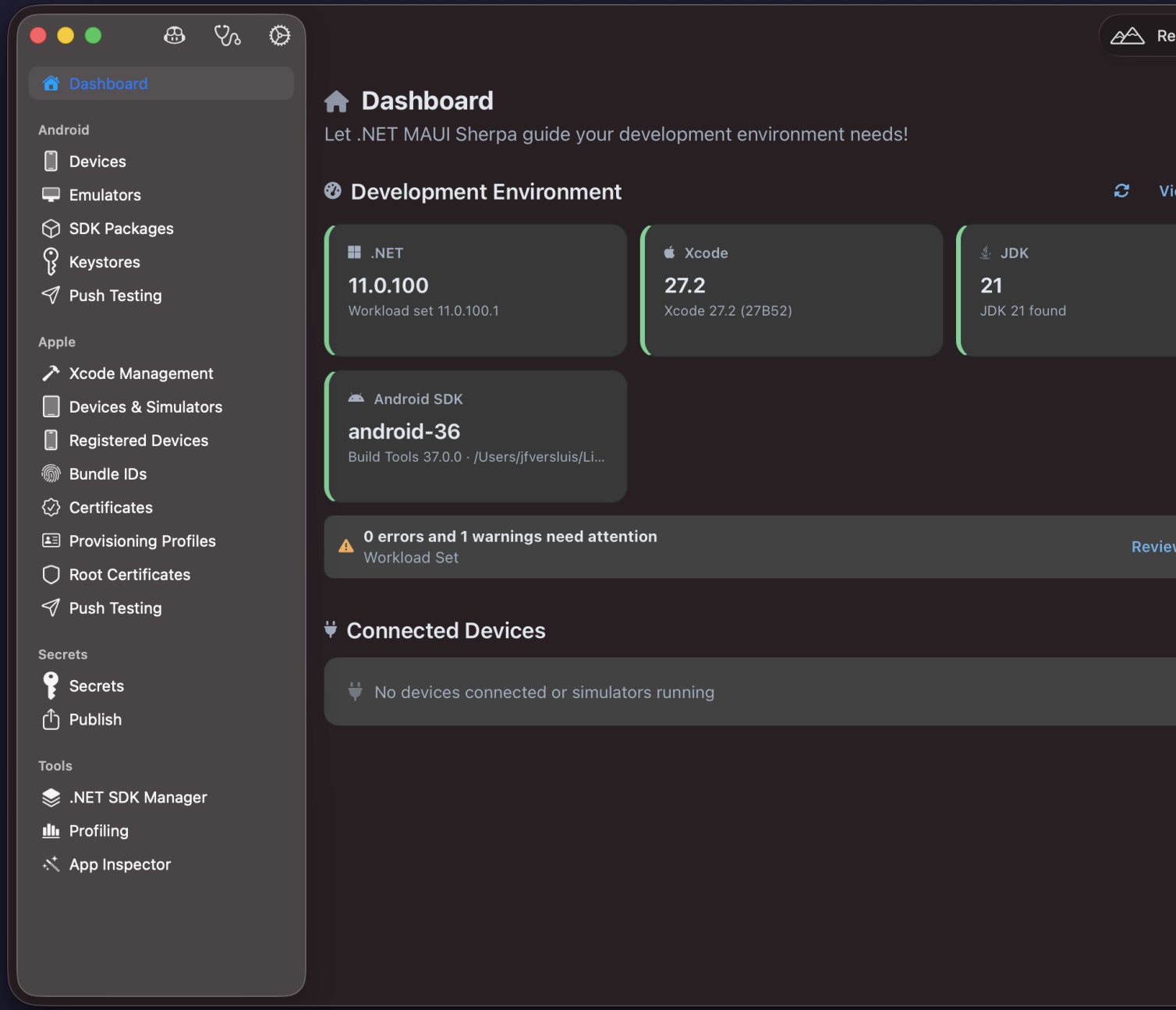
SDKs, emulators, simulators and certificates.

Browse a running app's visual tree, logs and network.

```
winget install Maui.Sherpa
brew install --cask redth/tap/maui-sherpa
```

redth.github.io/MAUI.Sherpa

Community tool by Redth (Jonathan Dick)



Skills for MAUI

Teach layouts, accessibility and Blazor Hybrid.

Show how to run and debug with DevFlow.

One command sets it all up in your editor.

```
# Preview what gets installed
maui ai init --env VsCode --dry-run

# Set up this project for AI
maui ai init --env VsCode --yes

# See what is installed, keep it fresh
maui ai status
maui ai update

# Or use the plugin marketplace
/plugin marketplace add dotnet/maui-labs
/plugin install dotnet-maui@dotnet-maui-labs
```

Works in the editor you already use

maui ai init sets up skills, agents and MCP for each one



Visual Studio

Agent mode uses your custom agents and MCP servers

`Agent (Preview)`



VS Code

Pick the MAUI agent and its tools in Copilot Chat

`--env VsCode`



Copilot CLI

The MAUI agent and DevFlow tools in your terminal

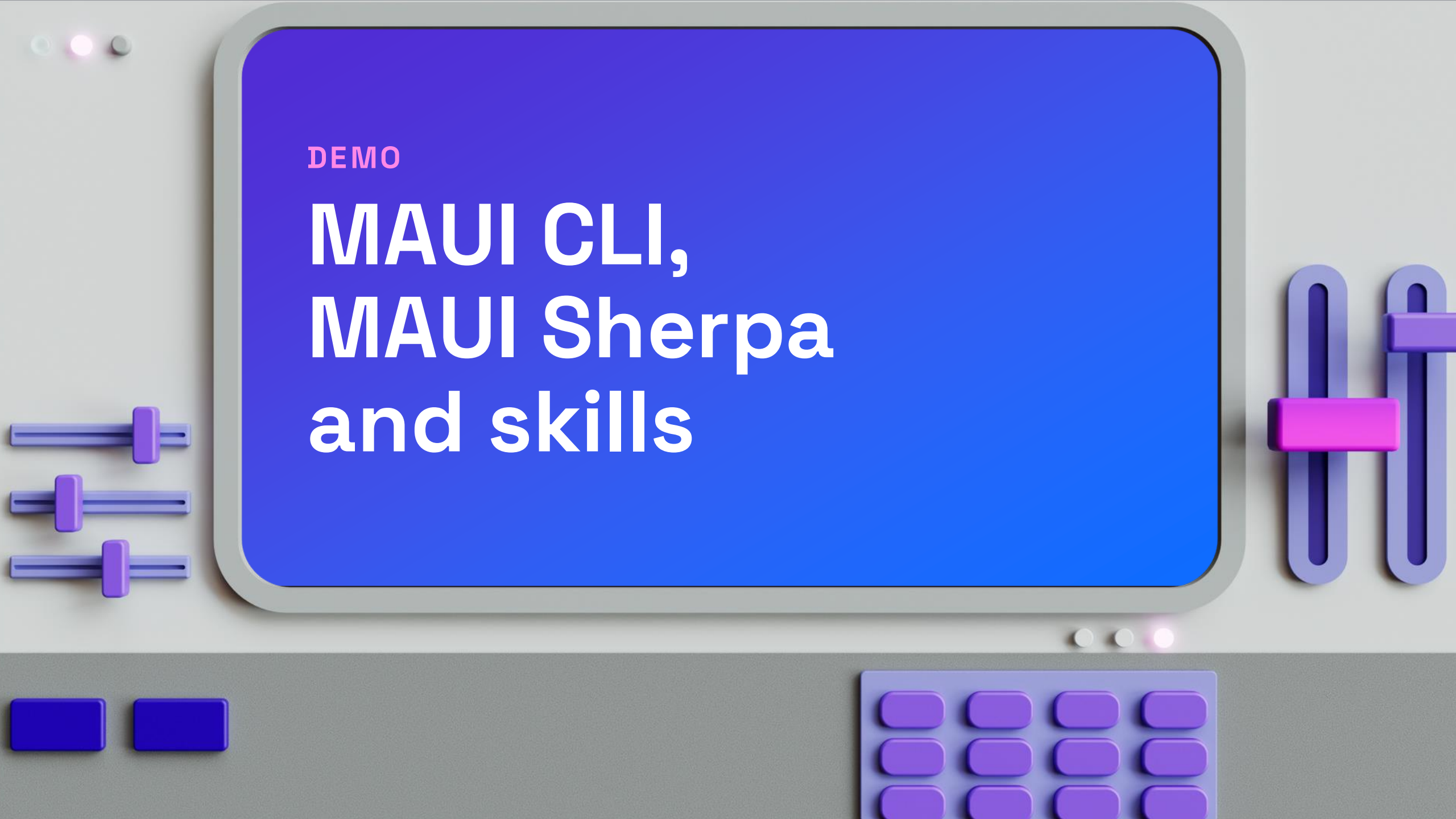
`maui ai init`



Copilot app

Plugins, plus canvases for the Inspector and devices

`/plugin install`


DEMO
MAUI CLI,
MAUI Sherpa
and skills

02 DevFlow



DevFlow gives the agent eyes and hands

Your AI talks to the running app through MCP

Your AI agent
`Copilot, Claude`

→

MCP server
`maui devflow mcp`

→

Broker
`devflow broker`

→

In-app agent
`Debug only`

What the agent can do in your app

Read the visual tree	Tap controls by name	Read properties	Stream logs
Watch network calls	Read app storage	Profile performance	Debug Blazor Hybrid

The visual tree beats a screenshot

DevFlow reads the app from the inside

Screenshot only

Guesses coordinates from pixels

Breaks when the layout shifts

DevFlow visual tree

Taps controls by name

Knows properties and state

Connect your app in three steps

```
// 1. Add the in-app agent
dotnet add package Microsoft.Maui.DevFlow.Agent --prerelease

// 2. Register it in MauiProgram.cs, Debug only
#if DEBUG
builder.AddMauiDevFlowAgent();
#endif

// 3. Start the broker, run the app, check the connection
maui devflow broker start
maui devflow list

// Or ask your agent: the maui-devflow-onboard skill does all of this
```

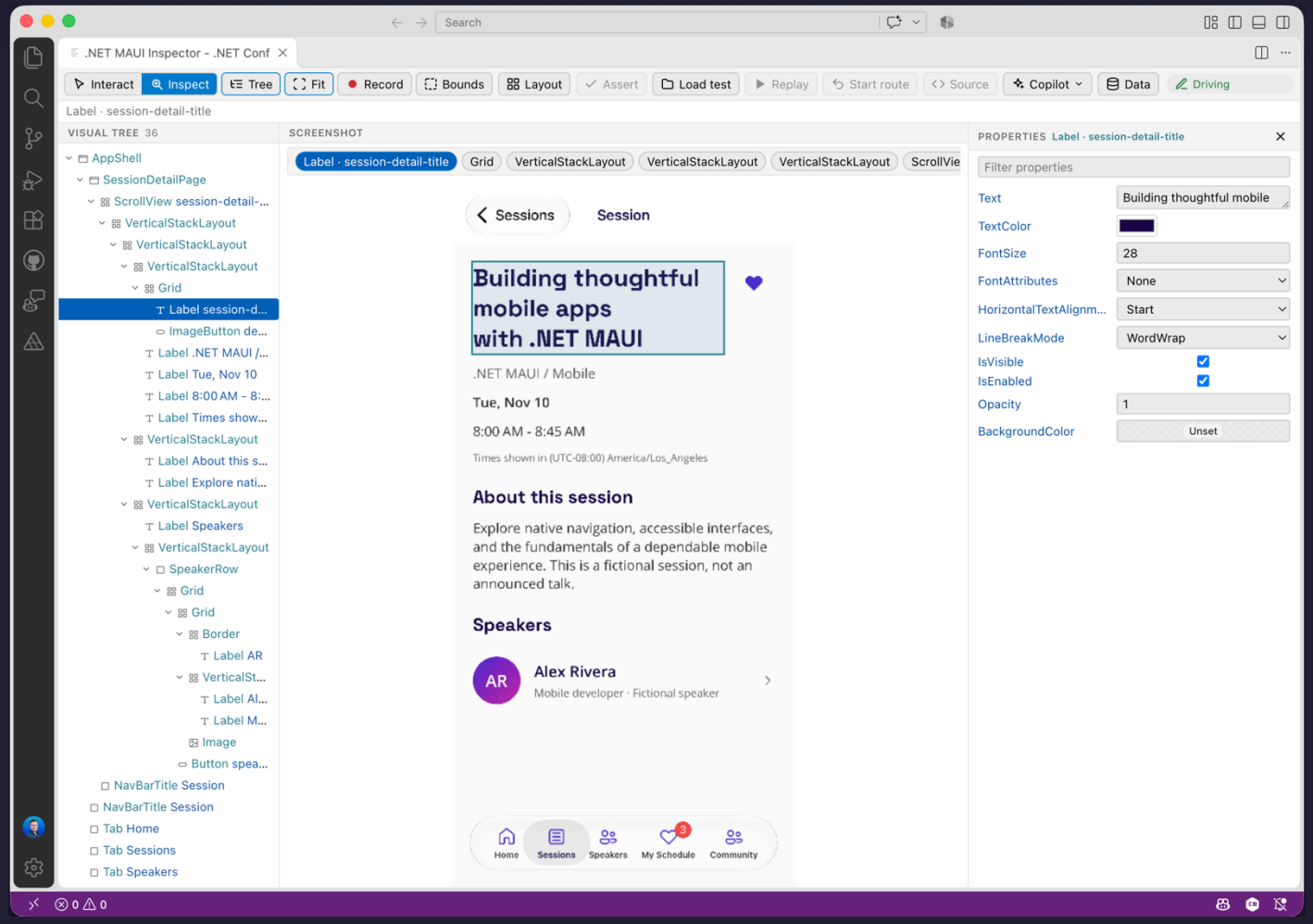

DEMO
Set up DevFlow
for an app

03 See it live



One live view for you and the AI

The Inspector shows what the agent sees



Visual tree
Every control on screen, by name

Live screenshot
Click a control to select it

Properties
Text, colors, layout and bindings

What the Inspector shows, and where

Open it wherever you already work

What you see

- Live screenshot of the app
- Visual tree of every control
- Properties, layout and bindings
- Logs, network and diagnostics

Where it opens

- Browser `localhost:19223/inspector`
- VS Code
- GitHub Copilot desktop canvas
- GitHub Copilot CLI canvas

Select a control, and the agent knows exactly which one you mean.

Mobile Canvas

A live simulator or emulator inside your editor.

You and Copilot drive the same device.

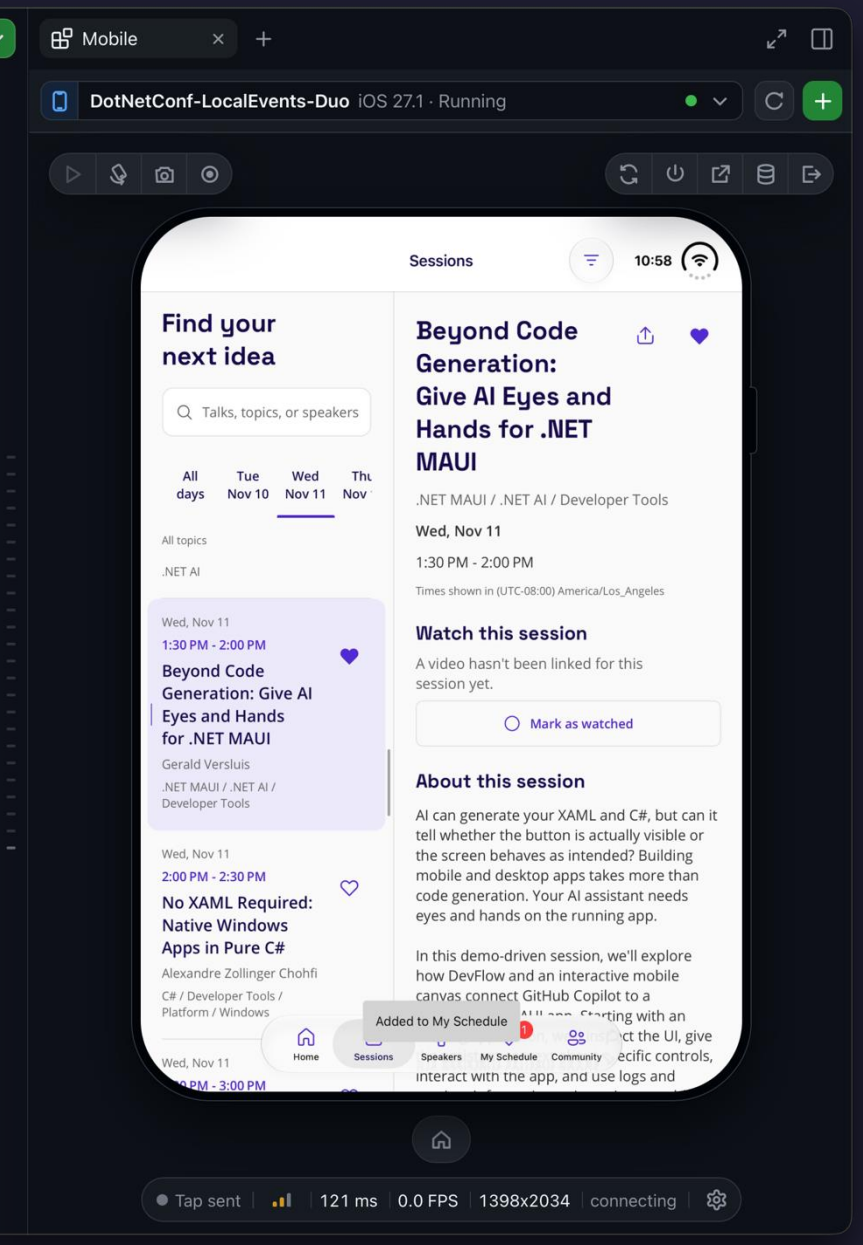
Reads the accessibility tree, not just pixels.

`VS Code: Mobile Canvas extension`

`/plugin install mobile-canvas`

github.com/Redth/mobile-canvas-ghcp

Community tool by Redth (Jonathan Dick)



04 All together



One loop ties the four tools together

Every step runs against the live app



Prompts that work

```
// Name the device, the symptom and the proof you want
“Run on Android. Save is missing on Profile. Fix it, then show me Save in the visual tree.”

// Point at the control instead of describing it
“Why is the control I selected in the Inspector 0 pixels wide?”

// Steer by name, not by coordinates
“Tap SaveButton, then read its IsEnabled and the bound command.”

// Ask it to check its own work
“Tap through sign-up again on iOS and check the logs for errors.”

// Set the boundaries
“Only change ProfilePage.xaml. Leave the view model alone.”
```

Habits that keep the agent useful

The agent does the legwork, you make the calls

Every session

- Start from a running app
- Ask for proof: the tree, logs or a replay
- Review every diff before you commit

Every team

- Commit the maui ai init setup
- Keep DevFlow in Debug builds only
- Test on real devices before you ship

Experimental previews: APIs and commands may change.

Get started

```
# 1. Install the MAUI CLI
dotnet tool install -g Microsoft.Maui.Cli --prerelease

# 2. Set up skills, agents and MCP in your project
maui ai init --env VsCode --yes

# 3. Ask your agent to add DevFlow, then open the Inspector
maui devflow broker start
http://localhost:19223/inspector/

# Docs, samples and issues
github.com/dotnet/maui-labs

# MAUI Sherpa and Mobile Canvas, by Redth
github.com/Redth
```

Resources

Everything from this session, in one place

- Tools, docs and samples
  `github.com/dotnet/maui-labs`
- MAUI CLI (.NET tool)
  `Microsoft.Maui.Cli`
- Download .NET 11
  `get.dot.net`

- MAUI Sherpa
  `redth.github.io/MAUI.Sherpa`
- Mobile Canvas
  `github.com/Redth/mobile-canvas-ghcp`

Try it, then tell us what breaks: open an issue on maui-labs.

Go deeper with .NET MAUI

Native apps for every platform, from mobile to Windows.

TUESDAY, NOVEMBER 10

BRK016
Bringing dotnet run, test, and watch to .NET MAUI
Jonathan Peppers
Tuesday • 4:30 PM PST

PRERECORDED

BRK033
Less Chat More Action: AI-Powered .NET MAUI Apps
Gerald Versluis
Prerecorded

BRK034
How We Brought CoreCLR to .NET MAUI and Mobile
Jonathan Peppers
Prerecorded

WEDNESDAY, NOVEMBER 11

BRK022
Building great Windows apps with WinUI and agents
Morten Nielsen
Wednesday • 11:30 AM PST

BRK024
CsWinRT 3.0: Modernizing the Windows Projections for .NET
Manodasan Wignarajah
Wednesday • 12:30 PM PST

BRK026
Beyond Code Generation: Give AI Eyes and Hands for .NET MAUI
Gerald Versluis
Wednesday • 1:30 PM PST

BRK027
No XAML Required: Native Windows Apps in Pure C#
Alexandre Zollinger Chohfi
Wednesday • 2:00 PM PST

Download

.NET 11

get.dot.net



Thank you

